

User Manual

Autonomous Agentic Development Platform

A comprehensive guide to TeamForge IDE — the intelligent development environment powered by six specialized AI agents, multi-provider AI chat, virtual file system, and autonomous workflow orchestration.

Version 1.0 • 2026

Table of Contents

1. Getting Started	3	4. AI Chat System	6
1.1 Overview	3	4.1 Chat Interface & Models	6
1.2 First Steps	3	4.2 Chat History & Sessions	6
1.3 Project Setup	3	5. Slash Commands	7
2. The IDE Interface	4	6. YOLO Mode	8
2.1 Top Bar & Sidebar	4	7. File Management & VFS	8
2.2 Editor & Chat Panel	4	8. Terminal & Build	9
2.3 Bottom Panel & Status Bar	4	9. Settings & Configuration	9
3. AI Agents	5	10. Keyboard Shortcuts	10
3.1 Agent Overview	5	11. Troubleshooting	9
3.2 Working with Agents	5		
3.3 Agent Collaboration	5		

Quick Reference

AI Agents

Atlas	Architect — System design, specs
Codey	Developer — Code implementation
Prism	Reviewer — Code review, security
Flux	Tester — Tests, edge cases
Blaze	DevOps — Deploy, CI/CD
Nova	PM — Coordination, tasks

AI Providers

Z-AI (GLM-4)	Default, best integration
NVIDIA NIM	80+ free models
OpenAI-Compatible	Custom endpoints

Top Commands

/edit	Edit code
/run	Execute commands
/fix	Fix errors
/commit	Git commit
/deploy	Deploy project

1. Getting Started

1.1 Overview

TeamForge IDE is a next-generation autonomous agentic development platform that integrates six specialized AI agents, a multi-provider AI chat system, a virtual file system (VFS), and intelligent workflow orchestration into a single cohesive environment. Unlike traditional IDEs, TeamForge delegates complex development tasks to AI agents that can plan, code, review, test, and deploy autonomously — while keeping you in full control.

At its core, TeamForge uses **GLM-4 models from Zhipu AI** (via the `z-ai-web-dev-sdk`) as the default AI provider. It also supports **NVIDIA NIM** (80+ free models) and any **OpenAI-compatible API endpoint**, giving you flexibility to choose the AI backbone that suits your needs.

1.2 First Steps

- 1 **Launch TeamForge IDE** — Open the application in your browser or desktop client. The main interface loads with the editor, chat panel, and file explorer ready.
- 2 **Sign in to AI providers** — Navigate to Settings to configure your AI provider. The default Z-AI (GLM-4) requires your Zhipu AI API key. Enter it to activate the AI agents and chat features.
- 3 **Create or open a project** — Use `Ctrl + N` to create a new file, or open an existing project folder from the file explorer in the sidebar.
- 4 **Start a chat** — Click the chat panel on the right side, type a message, and press `Enter`. The AI agent will respond based on your project context.
- 5 **Try a slash command** — Type `/help` in the chat input to see all available commands, or `/edit` to request an AI-powered code edit.

1.3 Project Setup

TeamForge IDE works with any project structure. To set up a project:

- **Open existing project** — Click the folder icon in the sidebar or use `Ctrl + O` to browse and open a directory. All files are loaded into the VFS.
- **Create from template** — Use the `/create_file` command or manually create files. The VFS supports all standard file types including `.js`, `.ts`, `.py`, `.rs`, `.go`, `.html`, `.css`, and more.
- **Initialize with a framework** — Ask any AI agent: *"Set up a Next.js project with TypeScript and Tailwind CSS"*. The agent will scaffold the project, create necessary files, and configure the environment.
- **Version control** — TeamForge integrates with Git. Use the `/commit` slash command or the terminal to manage version control operations.

Tip: The VFS (Virtual File System) is the backbone of TeamForge. All file operations happen in-memory first, giving AI agents instant access to your entire codebase without disk I/O bottlenecks. Changes are persisted to disk on save (`Ctrl + S`).

2. The IDE Interface

TeamForge IDE features a modern, panel-based layout designed for maximum productivity. The interface is divided into five primary zones, each serving a distinct purpose in the development workflow.

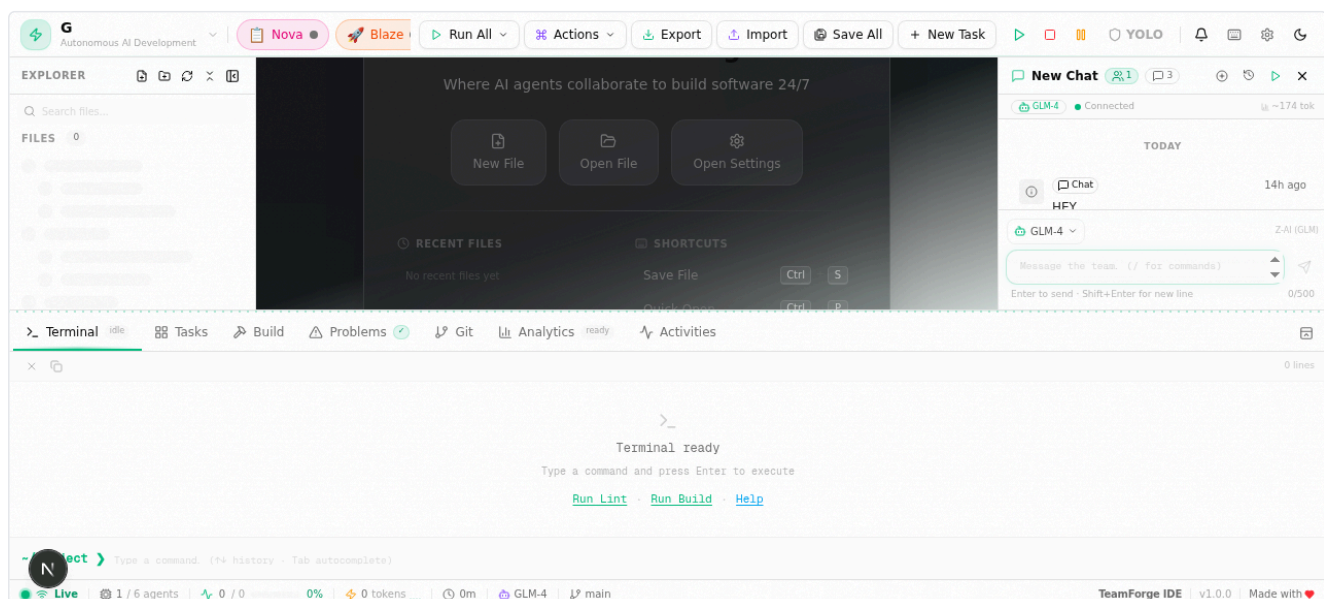


Figure 1: TeamForge IDE main interface showing all primary panels

2.1 Top Bar & Sidebar

The top bar provides global navigation and status information. It displays the **project name** (click to switch between recent projects), **agent status indicators** (colored dots showing which agents are active, idle, or processing), the **YOLO Mode toggle**, a **theme toggle** for light/dark switching, and the **settings gear icon**.

The left sidebar provides quick access to project resources:

Section	Description
File Explorer	Browse the project's VFS tree. Click any file to open it in the editor. Right-click for context actions (rename, delete, duplicate).
Search	Global search across all project files. Supports regex and case-sensitive modes. Results update in real-time as you type.
Git	View changed files, diffs, and commit history. Stage, unstage, and commit changes directly from the sidebar.
Agent Panel	Quick-access buttons to assign tasks to specific agents or view agent activity logs.

2.2 Editor & Chat Panel

The central **editor** supports multiple open files via tabs with syntax highlighting for 50+ languages, intelligent code completion powered by AI, inline error markers, and minimap navigation. Use `Ctrl + \` for split-view side-by-side editing.

The right-side **chat panel** is your primary interface for interacting with AI agents. It displays the current conversation thread, model selector, and agent assignment. You can drag the panel border to resize it. The chat supports markdown rendering, code blocks with syntax highlighting, and inline file references.

2.3 Bottom Panel & Status Bar

Component	Description
Terminal	Integrated terminal supporting bash/zsh/PowerShell. AI agents can execute commands here. Toggle with <code>Ctrl + `</code> .
Output	Displays build output, linting results, and agent task logs. Filterable by source.
Problems	Aggregated list of errors and warnings across the project, sorted by severity.
Status Bar	Shows current file language, line/column position, Git branch, AI model in use, and connection status.

3. AI Agents

3.1 Agent Overview

TeamForge IDE deploys six specialized AI agents, each designed for a specific phase of the software development lifecycle. All agents are powered by GLM-4 models and can be configured to use NVIDIA NIM or OpenAI-compatible providers.

Atlas ARCHITECT Designs system architecture, creates technical specifications, and plans project structure. Best for high-level design decisions, technology selection, and breaking down complex features into implementable components.	Codey DEVELOPER Writes and modifies code based on specifications. Handles core implementation — from scaffolding new files to implementing complex algorithms. Understands project context through the VFS and follows existing patterns.
Prism REVIEWER Performs code reviews, identifies bugs, security vulnerabilities, and style violations. Provides detailed feedback with line-level annotations and suggests specific fixes. Enforces coding standards across the codebase.	Flux TESTER Generates and runs test suites, identifies edge cases, and ensures code quality. Writes unit, integration, and end-to-end tests. Can execute tests via the terminal and analyze failures to suggest targeted fixes.
Blaze DEVOPS Manages deployment pipelines, Docker configurations, CI/CD workflows, and infrastructure. Builds containers, pushes images, configures environments, and handles release management.	Nova PROJECT MANAGER Coordinates tasks across agents, tracks progress, and manages timelines. Breaks down user stories, assigns work to the appropriate agent, monitors completion, and provides status reports.

3.2 Working with Agents

Assigning Tasks

You can direct tasks to specific agents in three ways:

- 1 **Direct mention in chat** — Prefix your message with the agent name: `@Codey Add input validation to the login form`. The named agent picks up the task immediately.

- 2 **Agent panel** — Click an agent in the sidebar Agent Panel to open a dedicated conversation with that agent.
- 3 **Auto-routing** — Simply describe your task without specifying an agent. Nova (the PM agent) analyzes the request and routes it to the best agent. Example: `Review the auth module for security issues` auto-routes to Prism.

3.3 Agent Collaboration

For complex multi-step tasks, agents collaborate through Nova's orchestration. When you request *"Build a user registration feature"*, Nova may: assign Atlas to design the registration flow and data schema; assign Codey to implement the frontend form and backend API; assign Flux to write and run tests; assign Prism to review the final code before merge. Each agent's output is visible in the chat panel, and you can intervene at any step.

Tip: In YOLO Mode, agents execute tasks without waiting for your approval between steps. Ideal for rapid prototyping but use with caution in production codebases.

4. AI Chat System

4.1 Chat Interface & Model Selection

The AI Chat is your primary interaction point with TeamForge's intelligence layer. It supports multi-turn conversations, code-aware context, and real-time streaming responses. The model selector at the top of the chat panel lets you switch between providers mid-conversation.

Supported AI Providers

Provider	Models	Setup	Notes
Z-AI (GLM-4)	GLM-4, GLM-4-Plus, GLM-4-Flash	API key in Settings	Default provider. Best agent integration. Powered by Zhipu AI's z-ai-web-dev-sdk. GLM-4-Flash for quick tasks.
NVIDIA NIM	80+ free models (Llama, Mistral, Qwen, etc.)	NVIDIA API key	Free tier available. Great for experimentation. Select from dropdown after configuring your key.
OpenAI-Compatible	Any OpenAI chat completions API endpoint	Base URL + API key	Works with vLLM, Ollama, LM Studio, Azure OpenAI, Together AI, Fireworks AI, and more.

How to Switch Models

- 1 Click the **model selector dropdown** at the top of the chat panel (shows the currently active model name).
- 2 Select a provider tab: **Z-AI**, **NVIDIA NIM**, or **Custom**.
- 3 Choose a specific model from the list. The chat instantly switches — no page reload needed.
- 4 Your conversation context is preserved when switching. The new model receives the full chat history.

4.2 Chat History & Session Management

TeamForge maintains a complete history of all chat sessions, organized by project and date. You can resume previous conversations or start fresh ones at any time.

- **New session** — Click the "+" button in the chat header or press `Ctrl + Shift + N`. Starts a clean conversation with no prior context.
- **Resume a session** — Click the clock icon in the chat header to view recent sessions. Select one to restore the full conversation history.
- **Pin important sessions** — Right-click any session and select "Pin" to bookmark it. Pinned sessions appear at the top of the history list.
- **Delete sessions** — Right-click and select "Delete" to remove a session permanently.

Context Awareness

The chat system automatically provides AI agents with relevant project context: **open files** in the editor are included in the context window; **selected code** is auto-referenced (also use `@filename` syntax); **project structure** from the VFS tree; and **terminal output** (errors, logs) when relevant.

Tip: For best results, keep relevant files open in the editor while chatting. This gives the AI maximum context. Use `@filename` to explicitly reference any project file in the VFS.

5. Slash Commands

Slash commands are powerful shortcuts that trigger specific AI-powered actions directly from the chat input. Type `/` to see the autocomplete menu, or type the full command with your request.

Command	Purpose	Description & Usage
<code>/help</code>	Show help	Displays the complete list of available slash commands with brief descriptions. Use when you forget a command name.
<code>/run</code>	Execute code	Runs the current file or a specified command. Example: <code>/run python main.py</code> executes in the terminal. The agent monitors output and reports results.

<code>/edit</code>	Edit code	Requests an AI-powered code edit. Describe the change: <code>/edit Add error handling to fetchData</code> . Shows a diff for review (or applies directly in YOLO Mode).
<code>/explain</code>	Explain code	Asks the AI to explain the selected code or current file. Provides line-by-line analysis, design rationale, and identifies potential issues.
<code>/fix</code>	Fix errors	Automatically diagnoses and fixes errors. The agent reads error messages, identifies root causes, and generates patches. Example: <code>/fix TypeScript errors in auth.ts</code>
<code>/refactor</code>	Refactor code	Restructures code without changing behavior. Supports renaming, extracting functions, simplifying conditionals, and design pattern application.
<code>/optimize</code>	Optimize perf	Analyzes code for performance bottlenecks and suggests or applies optimizations. Covers algorithmic complexity, memory usage, caching, and async patterns.
<code>/search</code>	Search code	AI-powered semantic search across the entire project. Understands intent. Example: <code>/search Where is user authentication handled?</code>
<code>/commit</code>	Git commit	Stages all changes and creates a commit with an AI-generated message based on the diff. Provide context: <code>/commit Include the new caching layer</code>
<code>/status</code>	Project status	Displays current project state: active agents, pending tasks, recent changes, test status, and errors/warnings.
<code>/create_file</code>	New file	Creates a new file in the VFS with AI-generated boilerplate. Example: <code>/create_file src/utils/logger.ts</code> . Infers template from extension and project context.
<code>/run_tests</code>	Execute tests	Discovers and runs the project's test suite. Analyzes results, reports failures with diagnostics, and suggests fixes. Supports Jest, pytest, Go test, and more.
<code>/deploy</code>	Deploy	Initiates deployment through the configured pipeline. Blaze (DevOps) handles build, containerization, and deployment. Supports Docker, cloud platforms, and custom scripts.

Tip: Chain commands using semicolons: `/edit Add validation; /run_tests; /commit` performs all three actions sequentially, each using the previous step's output.

6. YOLO Mode

What is YOLO Mode?

YOLO Mode is TeamForge's autonomous execution mode. When enabled, AI agents operate **without confirmation prompts** — they can modify files, execute terminal commands, run tests, and make project changes directly. The name reflects the "You Only Live Once" philosophy: maximum speed, minimal friction.

6.1 How to Enable

- 1 Click the **YOLO toggle switch** in the top bar. It turns green when active.
- 2 Alternatively, type `/yoLo` in the chat to toggle the mode.
- 3 A confirmation dialog appears the first time you enable it, explaining the implications. Accept to proceed.

6.2 What Changes

Behavior	Normal Mode	YOLO Mode
File modifications	Agent shows diff; you click "Apply"	Agent modifies files directly
Terminal commands	Agent shows command; you click "Run"	Agent executes immediately
Test execution	Agent proposes run; you approve	Agent runs tests and reads output
Git operations	Agent shows message; you confirm	Agent commits with generated message
Multi-agent tasks	Each handoff requires approval	Agents hand off seamlessly
Deployment	Agent prepares; you trigger	Agent deploys directly

Warning: YOLO Mode gives agents full write access to your project. Always review changes before committing to shared branches. Use YOLO Mode on feature branches and review the diff before merging.

7. File Management & VFS

7.1 Virtual File System (VFS)

The VFS is TeamForge's in-memory file layer that provides instant access to all project files for both you and the AI agents. It operates transparently — you interact with files normally, and the VFS handles synchronization between memory and disk.

- **Read** — Files are loaded into the VFS when opened or referenced. The VFS caches content for fast re-access. Agents use VFS reads to understand your codebase.
- **Write** — When you or an agent edit a file, the VFS updates in memory immediately. The editor reflects changes instantly. Press `Ctrl` + `S` to persist to disk.
- **Search** — The VFS supports full-text and regex search across all files. The `/search` command uses this for semantic search.
- **Watch** — The VFS watches for external file changes (e.g., from Git operations) and syncs automatically.

7.2 Creating & Editing Files

- **New file:** `Ctrl` + `N` opens a new untitled file. Save with `Ctrl` + `S` to name it and persist.
- **AI-created files:** Use `/create_file path/to/file.ext` to have an agent generate a file with appropriate boilerplate.
- **AI edits:** Use `/edit` or `@Codey` to request modifications. In normal mode, edits show as a diff preview before applying.
- **Multi-file edits:** Agents can modify multiple files in a single operation, maintaining consistency across related changes.

8. Terminal & Build

8.1 Integrated Terminal

TeamForge includes a fully functional terminal at the bottom of the interface. Toggle with `Ctrl` + ```. The terminal supports multiple sessions — click "+" to open additional instances. AI agents can execute commands when authorized (or automatically in YOLO Mode).

- **Manual execution** — Type any shell command and press `Enter`. Supports bash, zsh, and PowerShell.
- **AI-driven execution** — Use `/run <command>` to have an agent execute and monitor a command.
- **Build commands** — Trigger via `/run` (e.g., `/run npm run build`) or directly in the terminal.

8.2 Build & Deploy Workflow

- 1 **Build** — Run `/run npm run build` (or equivalent). Blaze can detect your project type and suggest the correct build command.
- 2 **Test** — Run `/run_tests` to execute the test suite. Flux analyzes failures and suggests fixes.
- 3 **Review** — Run `@Prism Review the latest changes` for a quality check before deploying.
- 4 **Commit** — Run `/commit` to stage and commit with an AI-generated message.
- 5 **Deploy** — Run `/deploy` and Blaze handles the rest: containerization, image push, and deployment.

9. Settings & Configuration

9.1 AI Provider Configuration

Open Settings via the gear icon in the top bar or `Ctrl` + `,`. The Settings panel is organized into tabs for each provider.

Z-AI (GLM-4) — Default Provider

- 1 Open Settings and navigate to the **AI Providers** tab.
- 2 Under **Z-AI**, enter your Zhipu AI API key in the designated field.
- 3 Select a default model: **GLM-4** (balanced), **GLM-4-Plus** (highest quality), or **GLM-4-Flash** (fastest).
- 4 Click **Test Connection** to verify your key. A green checkmark confirms connectivity.

NVIDIA NIM Setup

- 1 In Settings, select the **NVIDIA NIM** tab.
- 2 Enter your NVIDIA API key (get a free key at build.nvidia.com).
- 3 Browse the model catalog (80+ models) and select your preferred model.
- 4 Click **Test Connection**. NVIDIA NIM models are served via inference microservices.

OpenAI-Compatible Setup

- 1 In Settings, select the **Custom / OpenAI-Compatible** tab.
- 2 Enter the **Base URL** (e.g., `https://api.your-provider.com/v1`).
- 3 Enter your **API Key** for the endpoint.
- 4 Specify the **Model ID** (e.g., `meta-llama/llama-3-70b`).
- 5 Optionally adjust: temperature, max tokens, top_p.
- 6 Click **Test Connection** to verify the endpoint responds correctly.

9.2 Theme & Preferences

- **Theme toggle** — Switch between Light and Dark themes via the top bar icon or `Ctrl` + `Shift` + `T`.
- **Font size** — Adjust editor font size in Settings under **Editor**. Default: 14px.
- **Auto-save** — Enable/disable automatic file saving. When on, files save on focus change.
- **Terminal shell** — Choose your preferred shell (bash, zsh, PowerShell) under **Terminal** settings.

10. Keyboard Shortcuts

Action	Windows/Linux	macOS	Context
New file	Ctrl + N	Cmd + N	Global
Open file / folder	Ctrl + O	Cmd + O	Global
Save file	Ctrl + S	Cmd + S	Editor
Save all files	Ctrl + Shift + S	Cmd + Shift + S	Global
Find in file	Ctrl + F	Cmd + F	Editor
Find in project	Ctrl + Shift + F	Cmd + Shift + F	Global
Replace	Ctrl + H	Cmd + H	Editor
Toggle terminal	Ctrl + `	Cmd + `	Global
Toggle sidebar	Ctrl + B	Cmd + B	Global
Toggle chat panel	Ctrl + Shift + C	Cmd + Shift + C	Chat
Split editor view	Ctrl + \	Cmd + \	Editor
New chat session	Ctrl + Shift + N	Cmd + Shift + N	Chat
Send chat message	Enter	Enter	Chat
Newline in chat	Shift + Enter	Shift + Enter	Chat
Open Settings	Ctrl + ,	Cmd + ,	Global
Toggle YOLO Mode	Ctrl + Shift + Y	Cmd + Shift + Y	Global
Toggle theme	Ctrl + Shift + T	Cmd + Shift + T	Global
Close current tab	Ctrl + W	Cmd + W	Editor
Switch tabs	Ctrl + Tab	Cmd + Tab	Editor
Go to line	Ctrl + G	Cmd + G	Editor
Command palette	Ctrl + Shift + P	Cmd + Shift + P	Global
Undo / Redo	Ctrl + Z / Ctrl + Shift + Z	Cmd + Z / Cmd + Shift + Z	Editor
Quick open file	Ctrl + P	Cmd + P	Global

11. Troubleshooting

Issue	Possible Cause	Solution
AI agents not responding	API key missing, invalid, or expired	Open Settings and verify your API key under the active provider. Click "Test Connection" to check. If using Z-AI, ensure your Zhipu AI account has sufficient credits.
Slow AI responses	Large context window or high-demand model	Switch to GLM-4-Flash for faster responses. Close unnecessary files to reduce context size. For NVIDIA NIM, try a smaller model (e.g., Llama 3 8B instead of 70B).
Agent edits not applying	Normal mode requires manual approval	Check the chat panel for a pending diff preview. Click "Apply" to accept changes, or enable YOLO Mode for automatic application.
NVIDIA NIM connection fails	Invalid API key or network issue	Verify your NVIDIA API key at build.nvidia.com . Check firewall settings. Ensure the endpoint URL is correct in your configuration.
Custom endpoint not working	Incorrect Base URL or format mismatch	Ensure the Base URL ends with <code>/v1</code> and follows the OpenAI chat completions format. Test with <code>curl</code> first to confirm the endpoint is reachable.
File changes not persisting	Changes saved in VFS but not to disk	Press <code>Ctrl + S</code> to save the current file. Use <code>Ctrl + Shift + S</code> to save all files. Enable auto-save in Settings.
Terminal not responding	Process hung or terminal session crashed	Click the trash icon to terminate the current process. Open a new terminal session with the "+" button. Check if a long-running process is blocking.
Chat history missing	Session expired or storage cleared	Check the chat history panel (clock icon). Pinned sessions persist longer. Consider exporting important conversations.

Need more help? If you encounter an issue not listed here, try using the `/help` command in the chat, or describe your problem to Nova (the PM agent) who can guide you through diagnostics and connect you with the right agent for resolution.

11.1 Common Error Messages

Error	Meaning	Resolution
API_KEY_INVALID	The provided API key is not recognized by the provider	Double-check your key for typos. Regenerate the key from the provider dashboard if needed.
MODEL_NOT_FOUND	The selected model ID does not exist on the provider	Verify the model ID matches the provider's catalog. For NVIDIA NIM, check build.nvidia.com for available models.
CONTEXT_LENGTH_EXCEEDED	The conversation exceeds the model's context window	Start a new chat session to reset the context. Close unnecessary editor tabs to reduce the automatic context payload.
VFS_SYNC_ERROR	The virtual file system cannot sync with disk	Check file permissions. Ensure no other process has locked the files. Restart the IDE if the issue persists.

11.2 Performance Tips

- **Minimize open files** — Each open file adds to the AI context window. Close files you don't need for the current task to reduce token usage and improve response speed.
- **Use GLM-4-Flash for simple tasks** — Quick edits, small fixes, and short questions don't need the full GLM-4 model. Switch to Flash for 3-5x faster responses.
- **Be specific in agent requests** — Instead of `@Codey Fix the bug`, say `@Codey Fix the null reference error in UserService.ts line 42`. Specificity reduces agent iteration cycles.
- **Use YOLO Mode judiciously** — Enable YOLO Mode for well-defined tasks on feature branches. Disable it when exploring or when changes affect shared code.
- **Regular commits** — Use `/commit` frequently to create checkpoints. This makes it easy to revert if an agent makes unintended changes.